DemoStore — Prototype E-Commerce Web App

A fully functional prototype e-commerce web application demonstrating a secure, PCI DSS-compliant payment integration using **Stripe**, **Microsoft SQL Server**, and a complete **REST API** server — all built on **Django 5.x**.

Table of Contents

1. Purpose
2. High-Level Architecture
3. Tech Stack
4. Project Structure
5. Django Apps
   - Store App
   - Payments App
   - API App
6. Stripe Integration
7. SQL Server Database
8. REST API Server
9. Security
10. Test Suite (TDD)
11. Configuration
12. Getting Started
13. Sample Data

Purpose

DemoStore is a prototype application built to showcase how a modern Python/Django web app integrates three distinct backend systems:

- **Stripe** — for secure, server-side payment processing using PaymentIntents and webhooks
- **Microsoft SQL Server** — as the relational database backend (replacing Django's default SQLite)
- **Django REST Framework** — as a self-hosted REST API server exposing full CRUD access to all data, with interactive OpenAPI documentation

The application is intentionally scoped as a demonstration: it is production-ready in its patterns and security posture but is not a deployed commercial product.

High-Level Architecture

```
Browser
  │
  ├──→ Django Views (store & payments)
  │         │
  │         ├──→ SQL Server (via mssql-django / ODBC Driver 17)
  │         │         Tables: Product, Order, OrderItem, Payment, WebhookEvent
  │         │
  │         └──→ Stripe API (server-side PaymentIntent creation)
  │
  ├──→ Stripe.js / Elements (card data goes directly to Stripe — never to your server)
  │         │
  │         └──→ Stripe servers → redirect to /payments/succes/
  │
  ├──→ Stripe Webhook → /payments/webhook/  (async event confirmation)
  │
  └──→ REST API (/api/) — Django REST Framework
            │
            └──→ SQL Server (same database, same Django ORM)
                 Swagger UI:  /api/docs/
                 ReDoc:       /api/redoc/
```

Key design principle: **the server never handles raw card data**. Stripe Elements renders the card input in a hosted iframe and sends card details directly to Stripe's servers. Django only receives a `payment_intent_id` and verifies it server-side.

Tech Stack

Layer	Technology
Web framework	Django 5.x
Language	Python 3.11+
Payment provider	Stripe (Elements + PaymentIntents API)
Database	Microsoft SQL Server 2019+
ORM / DB driver	`mssql-django` + ODBC Driver 17 for SQL Server
REST API	Django REST Framework (DRF)
API documentation	`drf-spectacular` (Swagger UI + ReDoc)
Static files	WhiteNoise
Image handling	Pillow
Environment config	`python-decouple` (.env file)
Frontend	Bootstrap 5, Stripe.js
Local webhook testing	Stripe CLI

`requirements.txt` summary:

Package	Role
`Django>=5.2`	Core web framework
`stripe>=8.0.0`	Stripe Python SDK
`mssql-django>=1.4`	SQL Server database backend
`pyodbc>=5.0.0`	ODBC connectivity layer
`djangorestframework>=3.14.0`	REST API framework
`drf-spectacular>=0.27.0`	OpenAPI schema + Swagger/ReDoc UI
`whitenoise>=6.6.0`	Static file serving
`python-decouple>=3.8`	Environment variable management
`Pillow>=10.3.0`	Product image support

Project Structure

```
demoIntegrationPayment/
├── manage.py
├── requirements.txt
├── preferences.json            ← Currency settings (currency_code, currency_symbol)
├── .env                        ← Environment variables (not committed to VCS)
├── stripe-listen.bat           ← Start Stripe CLI webhook listener
├── stripe-start.bat            ← Start Stripe CLI + Django dev server
│
├── demostore/                  ← Django project package
│   ├── settings.py             ← All app settings, DB config, Stripe keys
│   ├── urls.py                 ← Root URL routing
│   ├── wsgi.py
│   └── asgi.py
│
├── store/                      ← Product catalog & session cart
│   ├── models.py               ← Product, Order, OrderItem
│   ├── views.py                ← List, detail, cart operations
│   ├── context_processors.py   ← Cart count + currency in every template
│   └── templates/store/
│       ├── base.html
│       ├── product_list.html
│       ├── product_detail.html
│       └── cart.html
│
├── payments/                   ← Stripe integration & webhooks
│   ├── models.py               ← Payment, WebhookEvent
│   ├── views.py                ← checkout, success, cancelled, webhook
│   └── templates/payments/
│       ├── checkout.html       ← Stripe Elements embedded
│       ├── success.html
│       └── cancelled.html
│
├── api/                        ← REST API (DRF + drf-spectacular)
│   ├── serializers.py
│   ├── views.py
│   └── urls.py
│
├── tdd/                        ← Test-Driven Development test suite
│   ├── test_models.py
│   ├── test_views.py
│   ├── test_context_processors.py
│   └── run_tests.ps1
│
├── fixtures/
│   └── demo_products.json      ← 5 sample products
│
└── sql/
    └── setup_db.sql            ← SQL Server database & login setup
```

Django Apps

Store App

Manages the product catalog, stock, and the session-based shopping cart.

Data Models

`Product` — items sold in the store

Field	Type	Description
`name`	CharField	Product name
`description`	TextField	Full description
`price`	DecimalField	Unit price
`image`	ImageField	Optional product photo
`stock`	PositiveIntegerField	Current inventory count
`available`	BooleanField	Whether the product is listed for sale

`Order` — a customer's purchase record

Field	Type	Description
`customer_name`	CharField	Buyer's full name
`customer_email`	EmailField	Buyer's email address
`status`	CharField	`pending` / `paid` / `failed` / `refunded`
`total_amount`	DecimalField	Order total
`created_at`	DateTimeField	Auto-set on creation

`OrderItem` — line items linking an Order to a Product

Field	Type	Description
`order`	ForeignKey	Parent order
`product`	ForeignKey	Product purchased
`quantity`	PositiveIntegerField	Units purchased
`unit_price`	DecimalField	Price at time of purchase (snapshot)

Views & Routes

URL	Description
`/`	Product grid — all available products
`/produit/<pk>/`	Individual product detail page
`/panier/`	Shopping cart (quantity controls, subtotals)
`/panier/ajouter/<pk>/`	Add item to cart (POST, stock-validated)
`/panier/supprimer/<pk>/`	Remove item from cart (POST)
`/panier/modifier/<pk>/`	Update item quantity (POST, atomic)

Stock is decremented using Django `F()` expressions — atomic database operations that prevent overselling under concurrent load.

Payments App

Handles Stripe PaymentIntent creation, checkout rendering, order fulfilment, and asynchronous webhook processing.

Data Models

`Payment` — tracks each Stripe transaction

Field	Type	Description
`order`	ForeignKey	Associated order
`stripe_payment_intent_id`	CharField	Stripe `pi_…` identifier
`stripe_charge_id`	CharField	Stripe `ch_…` identifier (post-payment)
`amount`	DecimalField	Amount charged
`currency`	CharField	ISO currency code
`status`	CharField	`pending` / `succeeded` / `failed` / `refunded`
`stripe_customer_email`	EmailField	Email from Stripe billing details
`failure_message`	TextField	Error details if payment failed

`WebhookEvent` — idempotent log of all Stripe webhook events

Field	Type	Description
`stripe_event_id`	CharField (unique)	Stripe `evt_…` identifier — prevents duplicate processing
`event_type`	CharField	e.g. `payment_intent.succeeded`
`payload`	JSONField	Full raw event body from Stripe
`processed`	BooleanField	Whether the event has been handled

Views & Routes

URL	Description
`/payments/checkout/`	Creates PaymentIntent, renders Stripe Elements form
`/payments/succes/`	Verifies payment, creates Order + Payment records, decrements stock
`/payments/annule/`	Cancellation page (cart is preserved)
`/payments/webhook/`	CSRF-exempt Stripe webhook endpoint

Webhook Events Handled

Stripe Event	Action
`payment_intent.succeeded`	Mark Payment and Order as `paid`
`payment_intent.payment_failed`	Mark Payment and Order as `failed`, store failure message
`charge.refunded`	Mark Payment and Order as `refunded`

API App

A full REST API giving programmatic access to every table. No additional server process is required — it runs on the same Django process as the web app.

See the REST API Server section for the full endpoint reference.

Stripe Integration

Payment Flow

```
1.  User fills cart
        ↓
2.  Click "Checkout" → GET /payments/checkout/
        ↓
3.  Server creates Stripe PaymentIntent
      (amount is calculated server-side — client cannot tamper with it)
        ↓
4.  Stripe.js renders secure card input (Elements, hosted iframe)
        ↓
5.  User enters card details and submits
        ↓
6.  Stripe.js sends card data DIRECTLY to Stripe servers
      (card data never touches Django)
        ↓
7.  Stripe redirects browser → /payments/succes/?payment_intent=<id>
        ↓
8.  Server calls Stripe API to verify PaymentIntent status
        ↓
9.  Creates Order, OrderItem, Payment records
      Stock decremented atomically via F() expressions
        ↓
10. Success page displayed with order reference
        ↓
11. Stripe sends webhook event → /payments/webhook/ (async confirmation)
```

> **3D Secure:** If the card issuer requires additional authentication, Stripe handles the 3DS challenge automatically before the redirect step.

Local Webhook Testing

The Stripe CLI forwards webhook events from Stripe to your local server. Two convenience scripts are provided:

Script	Purpose
`stripe-listen.bat`	Start the Stripe CLI webhook listener only
`stripe-start.bat`	Start both the Stripe CLI listener and the Django dev server

---

SQL Server Database

The application uses **Microsoft SQL Server** as its database backend via `mssql-django`, a community-maintained Django database backend built on `pyodbc`.

Connection Configuration

The backend is configured in `demostore/settings.py`:

```
DATABASES = {
    'default': {
        'ENGINE': 'mssql',
        'NAME': 'demodjango',
        'HOST': 'localhost',
        'PORT': '1433',
        'OPTIONS': {
            'driver': 'ODBC Driver 17 for SQL Server',
            'trusted_connection': 'yes',  # Windows Authentication
        },
    }
}
```

SQL Authentication is also supported: set `DB_USER`, `DB_PASSWORD`, and `DB_TRUSTED_CONNECTION=no` in `.env`.

Database Tables Created by Django Migrations

Table	App	Description
`store_product`	store	Product catalog
`store_order`	store	Customer orders
`store_orderitem`	store	Order line items
`payments_payment`	payments	Stripe payment records
`payments_webhookevent`	payments	Idempotent Stripe event log
`django_session`	Django core	Session-based cart storage
`django_admin_log`	Django admin	Admin action audit trail
`auth_*`	Django auth	User/group/permission tables

Initial Database Setup

Run `sql/setup_db.sql` in SSMS or `sqlcmd` to create the database and optional SQL login, then apply Django migrations:

```
py manage.py migrate
```

REST API Server

The API is served at `/api/` on the same Django process — no separate server is needed.

Interactive Documentation

Interface	URL	Description
Swagger UI	`http://127.0.0.1:8000/api/docs/`	Try API calls interactively
ReDoc	`http://127.0.0.1:8000/api/redoc/`	Clean read-only reference
OpenAPI schema	`http://127.0.0.1:8000/api/schema/`	Raw JSON schema

Pagination

All list endpoints are paginated (20 items per page). Supports `page` and `page_size` query parameters.

```
{
  "count": 42,
  "next": "http://127.0.0.1:8000/api/products/?page=2",
  "previous": null,
  "results": [ ... ]
}
```

Common Query Parameters

Parameter	Example	Description
`search`	`?search=shirt`	Full-text search on indexed fields
`ordering`	`?ordering=-created_at`	Sort ascending or descending
`page`	`?page=2`	Navigate paginated results

Endpoints Reference

Products — `/api/products/`

Method	URL	Description
`GET`	`/api/products/`	List all products
`POST`	`/api/products/`	Create a product
`GET`	`/api/products/{id}/`	Retrieve a product
`PUT`	`/api/products/{id}/`	Full update
`PATCH`	`/api/products/{id}/`	Partial update
`DELETE`	`/api/products/{id}/`	Delete a product

Orders — `/api/orders/`

Method	URL	Description
`GET`	`/api/orders/`	List all orders (with embedded line items)
`POST`	`/api/orders/`	Create an order
`GET`	`/api/orders/{id}/`	Retrieve an order
`PUT / PATCH`	`/api/orders/{id}/`	Update an order
`DELETE`	`/api/orders/{id}/`	Delete an order

Order Items — `/api/order-items/`

Method	URL	Description
`GET`	`/api/order-items/`	List all order items
`POST`	`/api/order-items/`	Create an order item
`GET / PUT / PATCH / DELETE`	`/api/order-items/{id}/`	Standard CRUD

Payments — `/api/payments/`

Method	URL	Description
`GET`	`/api/payments/`	List all payment records
`POST`	`/api/payments/`	Create a payment record
`GET / PUT / PATCH / DELETE`	`/api/payments/{id}/`	Standard CRUD

> No raw card data is ever stored — only Stripe identifiers (`pi_…`, `ch_…`).

Webhook Events — `/api/webhook-events/`

Method	URL	Description
`GET`	`/api/webhook-events/`	List all received webhook events
`GET / PUT / PATCH / DELETE`	`/api/webhook-events/{id}/`	Standard CRUD

HTTP Status Codes

Code	Meaning
`200 OK`	Request succeeded
`201 Created`	Resource created
`204 No Content`	Resource deleted
`400 Bad Request`	Validation error (field-level errors in response body)
`404 Not Found`	Resource does not exist
`405 Method Not Allowed`	HTTP method not supported on this endpoint

Security

Measure	Implementation
PCI DSS compliance	Card data handled exclusively by Stripe — never sent to or stored on Django
Amount integrity	`PaymentIntent` amount calculated server-side; client cannot tamper with it
Webhook signature	`stripe.Webhook.construct_event()` cryptographically verifies every webhook
Idempotent webhooks	Unique `stripe_event_id` constraint prevents duplicate order fulfilment
CSRF protection	Django CSRF middleware + tokens on all state-changing forms
SQL injection	Django ORM parameterised queries used throughout
Session security	`SESSION_COOKIE_HTTPONLY=True`, `SESSION_COOKIE_SAMESITE=Lax`
Secret management	All keys stored in `.env` via `python-decouple` — never hard-coded
Production headers	`SECURE_SSL_REDIRECT`, HSTS (1 year + subdomains + preload), `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`

Test Suite (TDD)

The `tdd/` directory contains a suite of Django `TestCase` tests covering the core application logic. All tests run against an isolated in-memory test database — no SQL Server connection is required for testing.

Test Files

File	Coverage
`test_models.py`	`Product`, `Order`, `OrderItem` model behaviour, defaults, validation, cascades
`test_views.py`	All store views: product list/detail, cart add/remove/update, stock validation
`test_context_processors.py`	`cart_count` and `currency` context processors

Running Tests

```
# From the project root
py manage.py test tdd

# Using the provided PowerShell script (saves results to tdd/results/)
.\tdd\run_tests.ps1
```

Configuration

Copy `.env.example` to `.env` and fill in all values before running the app.

```
# Django
SECRET_KEY=your-django-secret-key
DEBUG=True

# Database (Windows Authentication)
DB_NAME=demodjango
DB_HOST=localhost
DB_PORT=1433
DB_TRUSTED_CONNECTION=yes

# Database (SQL Authentication — alternative)
# DB_TRUSTED_CONNECTION=no
# DB_USER=admin
# DB_PASSWORD=yourpassword

# Stripe
STRIPE_PUBLIC_KEY=pk_test_...
STRIPE_SECRET_KEY=sk_test_...
STRIPE_WEBHOOK_SECRET=whsec_...
```

Currency

Edit `preferences.json` to change the display currency:

```
{
  "currency_code": "EUR",
  "currency_symbol": "€"
}
```

Getting Started

Prerequisites

Requirement	Notes
Python 3.11+	
Microsoft SQL Server 2019+	Express edition is sufficient
ODBC Driver 17 for SQL Server	Download from Microsoft
Stripe CLI	For local webhook forwarding
Stripe account	Free test-mode account

Setup

```
# 1. Create and activate virtual environment
python -m venv .venv
.\.venv\Scripts\Activate.ps1

# 2. Install dependencies
pip install -r requirements.txt

# 3. Configure environment
Copy-Item .env.example .env
# Edit .env with your Stripe keys and database settings

# 4. Create the database (run sql/setup_db.sql in SSMS first)
py manage.py migrate

# 5. Create a Django superuser (for /admin/)
py manage.py createsuperuser

# 6. Load sample products
py manage.py loaddata fixtures/demo_products.json

# 7. Start the application (Stripe CLI + Django dev server)
.\stripe-start.bat
```

The application will be available at `http://127.0.0.1:8000/`.

Sample Data

Five demo products are included in `fixtures/demo_products.json`:

Product	Price	Stock
T-shirt Classic	€19.99	50
Casquette Logo	€14.99	30
Mug Isotherme	€24.90	20
Carnet A5	€9.90	100
Tote Bag Coton	€12.50	0 (unavailable)

Use the **Stripe test card** `4242 4242 4242 4242` with any future expiry date and any 3-digit CVC to simulate a successful payment.

Pages & Templates

All pages extend `store/base.html`, which provides the Bootstrap 5 layout, navbar with live cart item count (injected by a context processor), and Django message alerts.

Route	Template	Description
`/`	`store/product_list.html`	Product grid with stock badges
`/produit/<pk>/`	`store/product_detail.html`	Product image, description, add-to-cart
`/panier/`	`store/cart.html`	Editable cart with subtotals and checkout button
`/payments/checkout/`	`payments/checkout.html`	Order summary + Stripe Elements card form
`/payments/succes/`	`payments/success.html`	Order confirmation with Stripe payment reference
`/payments/annule/`	`payments/cancelled.html`	Cancellation notice (cart preserved)
`/admin/`	Django admin	Full admin panel for all models
`/api/docs/`	Swagger UI	Interactive REST API documentation